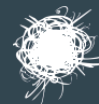


БypassAV, бесфайловая атака и AMSI (теория)

 habr.com/ru/articles/755034

August 16, 2023

Хабр



БypassAV, бесфайловая атака и AMSI (теория)

Разработка

Разработчики вредоносных программ постоянно ищут способ избежать обнаружения своих действий средствами защиты. Один из способов — обойти сканеры, используя обфускацию, шифрование, стеганографию и другие методы, чтобы антивирусному программному обеспечению было сложнее определить назначение «полезной нагрузки» или его загрузчика. Другой способ заключается в том, чтобы полностью избежать проверки кода вредоносного ПО или инструкций в памяти.

В этой статье рассмотрены типовые механизмы современных антивирусных средств, виды нагрузок, используемые злоумышленниками и специалистами в области информационной безопасности, а также методы обхода защиты. Эти знания помогут специалистам по информационной безопасности лучше понять логику атакующих и повысить защищенность своей организации.

Современные антивирусные средства (механизмы)

В настоящее время типовые средства обеспечения информационной безопасности имеют множество функций, которые поддерживают серьезный уровень контроля и фильтрации трафика. Базовым средством защиты на каждом рабочем месте в организациях остается средство антивирусной защиты (СABЗ или АВЗ).

Современные САВЗ являются сложной системой, подробности их работы не раскрываются, однако общие механизмы, лежащие в основе функционирования этих средств, известны.

При попадании на диск система защиты выдает новому файлу «уровень доверия» в зависимости от источника, разработчика, подписи и т.д. Исходя из уровня доверия, антивирусная система начинает сканирование файла до и после его запуска с соответствующей глубиной.

К механизмам анализа условно относят

1. Статический (aka Сигнатурный) анализ

Первый рубеж. Антивирусный «вендор» выявляет так называемую сигнатуру. *Сигнатура* вируса — это признак вредоносной программы (часть кода), по которому её можно «узнать» и однозначно определить. Такие сигнатуры вносятся в базу и с помощью сравнения исследуемого кода с элементами базы осуществляется поиск *плохиша*. Основная проблема в том, что обойти этот механизм очень просто: от небольшого изменения исходного кода и обфускации при компиляции, до шифрования уже готового исполняемого файла.

Анализ осуществляется с файлами, лежащими на диске до их исполнения в виртуальной памяти.

2. Динамический анализ

Sandbox или же песочница. Запуск исполняемых файлов в эмулируемой среде, имитирующей окружение целевой ОС. Это позволяет выявить все действия файла после потенциального запуска пользователем. Например, расшифрование криптоконтейнера с вредоносным кодом в теле программы или подключение к удаленному серверу. Данный вид анализа обойти сложнее, однако злоумышленники и исследователи в области ИБ находят методы обнаружения запуска в песочнице и не дают вредоносному участку кода выявить себя. Вот интересная [статья](#) от Positive Technologies.

Анализ осуществляется с файлами, лежащими на диске до их исполнения в виртуальной памяти или во время запуска пользователем.

3. Механизм распаковки

Является вспомогательным инструментом для динамического анализа. Злоумышленники часто упаковывают свой вредоносный код в контейнер путем различных преобразований для обхода статического и динамического обнаружения. Контейнеры с вредоносным кодом создаются вручную или с помощью различных упаковщиков или архиваторов. Сотрудники антивирусных компаний собирают все известные методы упаковки в единый механизм распаковки, который и позволяет выявить вредоносный код до запуска в песочнице. Метод экономит ресурсы и позволяет обнаружить большую часть опубликованных (с инструментами

автоматизации) методов сокрытия. Данный механизм уязвим, так как его обход заключается в создании метода упаковки на основе небольшого видоизменения любого из публичных методов.

Анализ осуществляется при попадании файла на диск до его исполнения в виртуальной памяти или во время запуска пользователем.

4. Поведенческий (ака эвристический) анализ

Иногда еще называют методом статистического оценивания потенциально вредоносного поведения, что говорит само за себя. Файлу и созданному им процессу выдается «**порог подозрительного поведения**». После запуска, если первые два механизма не обнаружили ничего вредоносного, антивирусное средство оценивает поведение кода в виртуальной памяти системы. Например, вызов нетипичных системных функций, подключение сторонних библиотек, самокопирование, внедрение в память других процессов, инициация удаленного подключения и т.д. Если количество подозрительных действий процесса **больше порога**, то принимается решение о том, что это **вредоносная программа**. Данный механизм является одним из самых перспективных с точки зрения систем защиты и сложных со стороны обхода обнаружения.

Анализ осуществляется с запущенным в виртуальной памяти машины кодом исследуемого процесса.

Почему же не использовать глубокий анализ для всех входящих файлов сразу? Ограничены ресурсы! Антивирусное средство не может себе позволить забрать большую часть ресурсов пользователя, что вынуждает осуществлять градацию по степеням доверия кода, поступившего на вход системы.

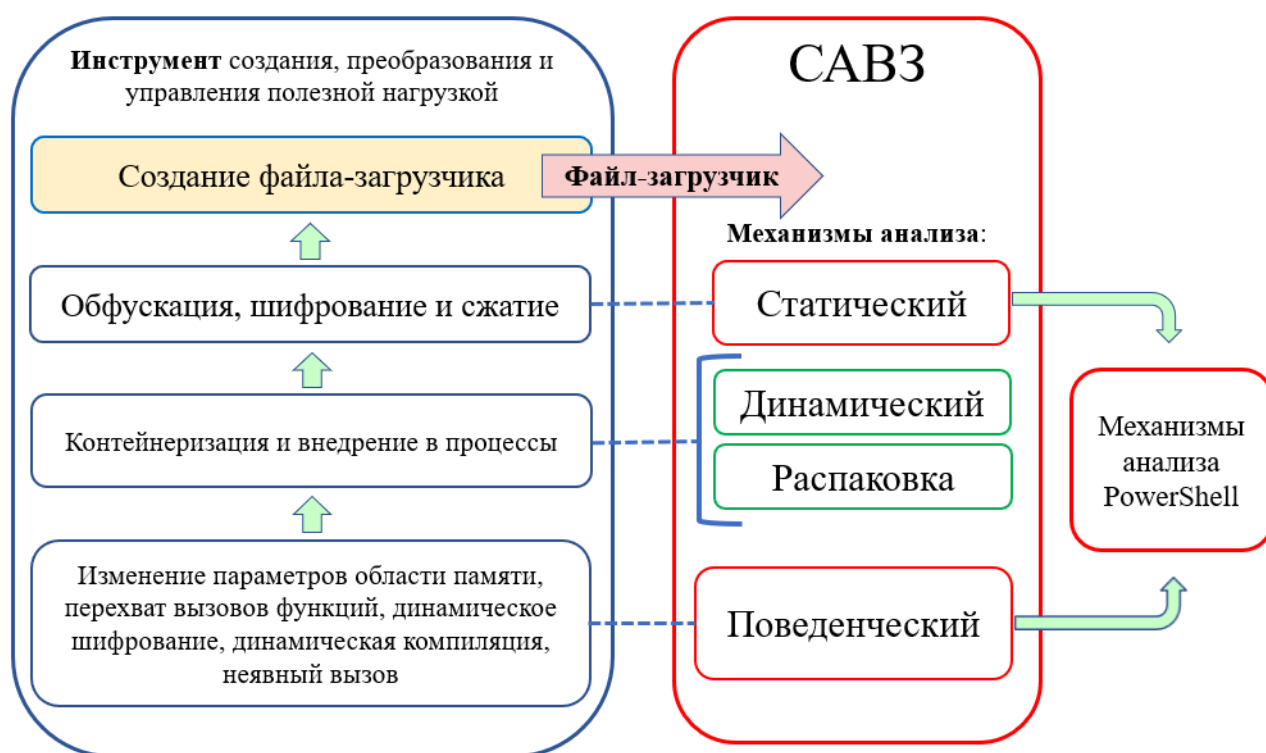
Типы угроз (файловые и бесфайловые)

В данной статье разделим все угрозы на две категории: файловые и бесфайловые. Файловые нам хорошо известны. Они попадают в систему через почту в половине случаев или через другой сетевой трафик (веб-интерфейс, любые другие протоколы передачи файлов). Их в свою очередь можно разделить на исполняемые .exe, .dll (бинарные) или неисполняемые, текстовые файлы, скрипты и т.д. К неисполняемым нагрузкам в последнее время повышенный интерес, ведь они являются основой для бесфайловых атак.

Бестелесное или [бесфайловое вредоносное ПО](#) — это такое ПО, которое не сохраняется на жестком диске в виде исполняемого файла, а загружается напрямую в виртуальную память. Такие зловреды опасны тем, что практически не оставляют следов в зараженной системе, поэтому их довольно сложно обнаружить. Так можно осуществить запуск Powershell кода сразу в процессе интерпретатора в лице

системной службы powershell.exe удаленно или из другого процесса. Они находятся в оперативной памяти системы, реестре и других непостоянных областях хранения, что затрудняет их обнаружение с помощью обычного антивирусного программного обеспечения. Главное условие — код не должен попасть на жесткий диск, чтобы избежать динамического анализа.

Почему мы рассматриваем бесфайловые угрозы? Потому что злоумышленники все чаще используют их для [атаки](#). На примере тех же powershell-нагрузок в виде файлов-скриптов или бесфайловой one-liner команды вредоносный код не подвергается динамическому анализу и распаковке. Проверка осуществляется только статическим и поведенческим анализом. И для достижения их цели они разрабатывают методы обхода только этих механизмов. Ниже продемонстрирован порядок создания необнаруживаемого ВПО и методы обхода соответствующих механизмов анализа.



Порядок создания необнаруживаемого ВПО и методы обхода соответствующих механизмов анализа

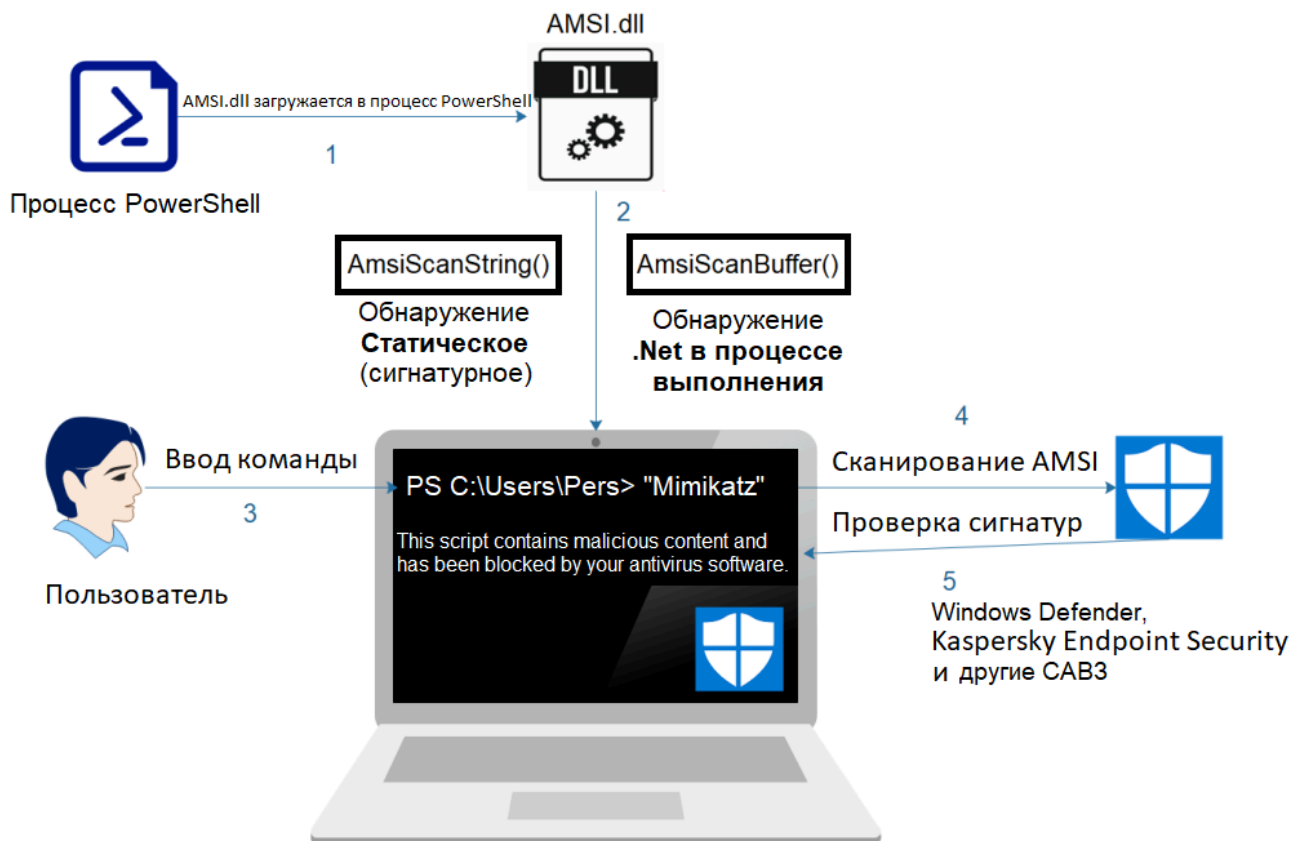
По такому принципу осуществляется анализ антивирусного средства на целевом хосте. На этапе изучения атакуемой системы злоумышленники моделируют среду для исследования и разработки подходящих методов обхода для конкретных средств защиты и их окружения. Обход этих средств — основа сразу трех этапов целевой атаки на объект: исследование модели целевой системы со средствами защиты, эксплуатация и закрепление.

Методы обхода [публикуются](#), но сразу же теряют актуальность. Для нагрузок Powershell часто используется стратегия «оживления» известных методов обхода и их реализация при реальной атаке. Коды публичных методов обхода защиты в кратчайший срок вносятся в базы антивирусных средств. Ведь само решение обойти защиту — это вредоносное действие.

Бypass AMSI

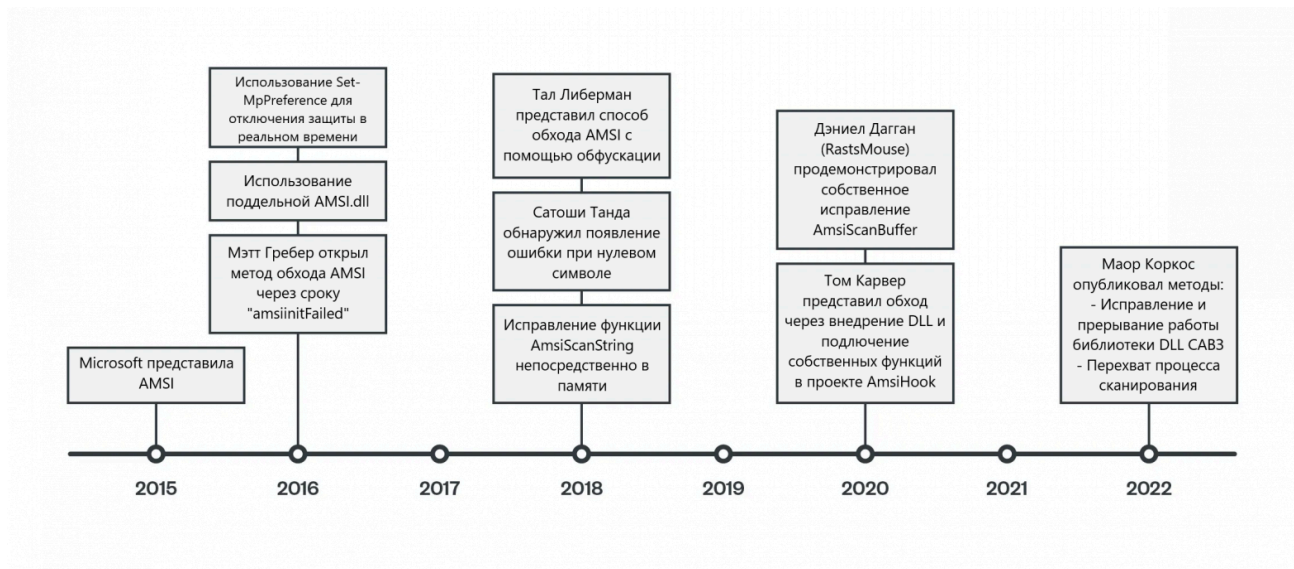
Существует большое количество статей с описанием метода обхода защиты от файловых нагрузок. Но как мы говорили раньше, неисполняемые файлы подвергаются меньшему анализу, что привлекает злоумышленников. На примере Powershell-загрузчиков рассмотрим механизм защиты в операционных системах семейства Windows. Интерфейс Microsoft Antimalware Scan. AMSI, представленный в 2015 году, предоставляет программному обеспечению свое внутреннее API для взаимодействия со средствами защиты, запрашивая сканирование файлов, памяти или потоков на наличие вредоносных полезных нагрузок методами сигнатурного и поведенческого анализа независимо от разработчика CAB3. Если стороннее антивирусное средство не установлено, то по умолчанию библиотека `amsi.dll` взаимодействует с Windows Defender.

AMSI является «мостом» между антивирусным средством и процессом Powershell. Средство AV3 осуществляет анализ скрипта функцией `Amsiscanstring()`, если он лежит на диске или после запуска пользователем, но до его передачи на исполнение процессу Powershell. А поведенческий анализ выявляет вредоносную активность после его запуска функцией `Amsiscanbuffer()`.



Со дня появления AMSI злоумышленники (и исследователи безопасности) создавали инструменты для обхода или отключения AMSI. Корпорация Майкрософт и поставщики антивирусных средств предпринимают шаги по блокированию [некоторых подходов](#), используемых для обхода AMSI, но злоумышленники продолжают приспосабливаться. Наиболее эффективными методами обхода являются различные варианты патчинга функций библиотеки AMSI.dll в памяти для недопущения проверки входящих скриптов антивирусным средством. Сам код таких обходов детектируется механизмами статического анализа, однако путем глубокой обфускации и многочисленных тестирований злоумышленники достигают своих целей и разрушают «мост», не давая антивирусному средству дойти до их основной нагрузки, передаваемой на уже полностью незащищенный процесс powershell.

Исследователи в области ИБ находят новые методы и публикуют их. Несомненно, данный код уже внесен в базы антивирусных средств, однако после глубокой обфускации и других преобразований над ними они встают [на вооружение](#) у злоумышленников.



Известные методы обхода AMSI

При анализе современных инцидентов наблюдают нагрузки, использующие методы отключения AMSI 2016 года, что говорит о применимости стратегии «оживления», не говоря о поиске и создании новых путей обхода.

Инструменты обфускации

Самые популярные инструменты автоматизации обфускации:

- <https://github.com/danielbohannon/Invoke-Obfuscation>
- <https://github.com/CCob/SharpBlock>
- <https://github.com/klezVirus/Chameleon>
- <https://github.com/tokyoneon/Chimera>

Однако их преобразования имеют паттерн, который хорошо известен разработчикам антивирусных средств. Часто их неправильное использование приводит к потере работоспособности исходного скрипта. Злоумышленники и исследователи берут их за основу. Также создаются и комбинируются собственные средства преобразования загрузчика, полезной нагрузки или кода отключения AMSI.

Но даже при использовании открытых инструментов напрямую получают **обфусцированный код powershell загрузчика любой C2 системы**, который пройдет механизм сигнатурного и поведенческого анализа ДО передачи основной нагрузки. А если вместе с ним был осуществлен патчинг `amsi`, то дальнейший анализ для данного процесса будет невозможен :)

Выводы

Итак, мы рассмотрели принципы работы современных САВЗ, упомянули методы их обхода и остановились подробнее на одном актуальном векторе атаки. Он заключается в применении powershell-загрузчиков Command and Control систем, обработанных путем глубокой обфускации и добавления кодов отключения механизмов защиты, например AMSI. Необходимо учитывать данные подходы при построении системы защиты от сетевого периметра до конкретного рабочего места в организации. Необходимо быть на шаг впереди, ведь злоумышленники продолжают разрабатывать новые инструменты для реализации известных векторов атак.